Cloud Computing — Project Compendium

Academic Year 2025–26 | 17 Project Topics

#01 Carbon-Aware Deep Reinforcement Learning Scheduler for Kubernetes Po Placement

PROBLEM STATEMENT

Existing Kubernetes schedulers ignore energy and carbon implications during pod placement, leading to inefficient resource consumption under fluctuating workloads. A DRL-based scheduler is needed that jointly minimizes SLA violations, node overload, and estimated carbon emission.

TOOLS & TECHNOLOGIES

- Kubernetes (custom scheduler framework)
- Python + PyTorch / TensorFlow (DRL model)
- Prometheus & Grafana (metrics & monitoring)
- Carbon Intensity API (real-time carbon data)
- Minikube / Kind (local cluster)
- Locust / k6 (workload generation)

#02 Intelligent Pod Scheduling

PROBLEM STATEMENT

Default Kubernetes scheduling may not optimize for latency or resource efficiency. A custom scheduler should be designed and its performance compared against the default.

TOOLS & TECHNOLOGIES

- Kubernetes Scheduler Extender / custom scheduler
- Go or Python (scheduler logic)
- Prometheus & Grafana (performance monitoring)
- kubectl & Helm (deployment)
- Locust / k6 (load testing)

#03 Zero Trust Kubernetes

PROBLEM STATEMENT

The default Kubernetes trust model is risky, exposing services to lateral movement attacks. RBAC and fine-grained network policies must be implemented to enforce zero-trust principles.

TOOLS & TECHNOLOGIES

- Kubernetes RBAC (Role-Based Access Control)
- Calico / Cilium (network policy enforcement)
- Open Policy Agent (OPA) / Gatekeeper
- Falco (runtime security monitoring)
- Keycloak / Dex (identity & auth)
- kube-bench (CIS benchmark auditing)

#04 ML Model Deployment on Kubernetes

PROBLEM STATEMENT

Manual ML model deployment is error-prone and hard to scale. The goal is to containerize and deploy an ML model on Kubernetes with a REST API endpoint.

TOOLS & TECHNOLOGIES

- Docker (containerization)
- Kubernetes (orchestration)
- FastAPI / Flask (model serving API)
- Scikit-learn / TensorFlow / PyTorch (ML model)
- KServe / Seldon Core (ML serving)
- Helm (packaging & deployment)

#05 Edge–Cloud Placement Decision Engine

PROBLEM STATEMENT

Deciding whether to execute workloads at the edge or in the cloud is non-trivial. Placement logic based on latency and resource availability must be developed.

TOOLS & TECHNOLOGIES

- KubeEdge / OpenYurt (edge-cloud framework)
- Python (decision engine logic)
- Prometheus (resource metrics collection)
- MQTT / gRPC (edge communication)
- Kubernetes (cloud orchestration)
- Grafana (latency & placement dashboard)

#06 High Availability Aware Pod Replication Controller for Stateful Application

PROBLEM STATEMENT

Stateful cloud applications face service interruptions during node failures because replication decisions are not topology-aware. An HA-aware replication controller must place replicas intelligently across failure domains.

TOOLS & TECHNOLOGIES

- Kubernetes StatefulSets & Pod Topology Spread
- Custom Controller (Go / Python via controller-runtime)
- etcd (state management)
- Prometheus & Alertmanager (failure detection)
- Chaos Mesh / LitmusChaos (fault injection testing)
- Helm (deployment packaging)

#07 Self-Healing Kubernetes Infrastructure using Predictive Node Failure Detection

PROBLEM STATEMENT

Traditional Kubernetes reacts only after node failure occurs. A predictive self-healing framework is needed that analyzes node telemetry, forecasts failures, and proactively migrates pods before outages happen.

TOOLS & TECHNOLOGIES

- Prometheus + Node Exporter (telemetry collection)
- Python + scikit-learn / LSTM (failure prediction model)
- Kubernetes Eviction API (pod migration)
- Grafana (visualization & alerting)
- Chaos Mesh (node failure simulation)
- Argo Workflows (remediation orchestration)

#08 Scalable Multi-Cluster Kubernetes Federation for Disaster Recovery on AV Free Tier

PROBLEM STATEMENT

Single-cluster deployments are vulnerable to complete service outages. A multi-cluster federated Kubernetes architecture across lightweight AWS nodes is needed to evaluate failover, synchronization delay, and disaster recovery.

TOOLS & TECHNOLOGIES

- AWS EC2 Free Tier (t2.micro / t3.micro nodes)
- KubeFed / Liqo / Admiralty (federation layer)
- kubeadm / k3s (lightweight cluster setup)
- Velero (backup & restore)
- Prometheus & Thanos (multi-cluster monitoring)
- Route 53 / MetalLB (traffic failover)

#09 Predictive Auto-Scaling

PROBLEM STATEMENT

Reactive scaling causes SLA violations because it responds too late to workload spikes. Workload prediction must be used to scale proactively ahead of demand.

TOOLS & TECHNOLOGIES

- Kubernetes HPA & KEDA (scaling controllers)
- Prometheus (metrics source)
- Python + Prophet / ARIMA / LSTM (forecasting model)
- Custom Metrics Adapter (feed predictions to HPA)
- Locust / k6 (workload simulation)
- Grafana (scaling visualization)

#10 FaaS Platform Deployment (Serverless Functions)

PROBLEM STATEMENT

Always-on applications waste compute resources during idle periods. A Function-as-a-Service platform should be deployed to enable serverless, event-driven execution.

TOOLS & TECHNOLOGIES

- OpenFaaS / Knative / Fission (FaaS platform)
- Kubernetes (underlying orchestration)
- Docker (function containerization)
- Prometheus & Grafana (function monitoring)
- NATS / Kafka (event triggers)
- Helm (platform deployment)

#11 Serverless Workflow Orchestration

PROBLEM STATEMENT

Multi-step applications need reliable orchestration across functions. A serverless workflow engine must chain functions together with error handling and state management.

TOOLS & TECHNOLOGIES

- Argo Workflows / Temporal (workflow orchestration)
- Knative / OpenFaaS (function runtime)
- Kubernetes (execution environment)
- Apache Kafka / NATS (inter-function messaging)
- Prometheus & Grafana (workflow monitoring)
- Helm (deployment management)

#12 ML Model Drift Detection and Continuous Retraining

PROBLEM STATEMENT

ML models degrade over time as data distributions shift. A drift detection system must monitor model performance and trigger automatic retraining when degradation is detected.

TOOLS & TECHNOLOGIES

- Evidently AI / Alibi Detect (drift detection)
- MLflow / Kubeflow Pipelines (retraining orchestration)
- Python + scikit-learn (model training)
- Kafka / Airflow (data pipeline)
- Kubernetes (deployment environment)
- Grafana (drift metrics dashboard)

#13 Intelligent Resource Allocation for CPU and Memory Optimization in Kubernetes

PROBLEM STATEMENT

Static CPU and memory requests lead to overprovisioning or underutilization. An intelligent controller must dynamically tune pod resource requests based on real-time utilization patterns.

TOOLS & TECHNOLOGIES

- Kubernetes VPA (Vertical Pod Autoscaler)
- Custom Controller (Python / Go via controller-runtime)
- Prometheus + metrics-server (utilization data)
- Grafana (resource efficiency dashboards)
- Locust / k6 (variable workload generation)
- kubectl (resource patching & management)

#14 Continuous ML Pipeline Automation

PROBLEM STATEMENT

ML models require frequent updates as new data arrives. The training and deployment pipeline must be fully automated to reduce manual effort and deployment lag.

TOOLS & TECHNOLOGIES

- Kubeflow Pipelines / MLflow (pipeline orchestration)
- Kubernetes (execution platform)
- Docker (model containerization)
- GitHub Actions / ArgoCD (CI/CD integration)
- FastAPI / KServe (model serving)
- Prometheus & Grafana (pipeline monitoring)

#15 Cold Start Optimization for Serverless Functions

PROBLEM STATEMENT

Cold start delays in serverless environments significantly impact response latency for end users. Techniques must be developed to reduce or eliminate cold start overhead.

TOOLS & TECHNOLOGIES

- Knative / OpenFaaS (serverless runtime)
- Kubernetes (cluster environment)
- Python / Go (optimized function runtimes)
- Prometheus (cold start latency measurement)
- Grafana (latency profiling dashboard)
- Locust / k6 (cold start simulation under load)

#16 Resource Efficient Multi-Tenant Kubernetes Infrastructure using Virtual Clusters

PROBLEM STATEMENT

Traditional multi-tenant environments require multiple dedicated clusters, causing high resource overhead and cloud cost. A vCluster-based infrastructure must consolidate tenants while preserving isolation.

TOOLS & TECHNOLOGIES

- vCluster (virtual cluster technology)
- Kubernetes (host cluster)
- Helm (vCluster deployment)
- Prometheus & Grafana (per-tenant monitoring)
- k6 / Locust (tenant workload simulation)
- kubectl & RBAC (tenant access control)

#17 Performance Comparison: Native vs Virtual Kubernetes Clusters under Bursty Workloads

PROBLEM STATEMENT

There is limited empirical analysis on how virtual clusters behave compared to full native clusters under increasing tenant traffic. A comparative framework must measure startup latency, CPU overhead, scheduling delay, and throughput.

TOOLS & TECHNOLOGIES

- vCluster (virtual cluster setup)
- Kubernetes (native cluster baseline)
- k6 / Locust (bursty workload generation)
- Prometheus + metrics-server (performance data)
- Grafana (comparative dashboards)
- Python / Jupyter (statistical analysis & plotting)